

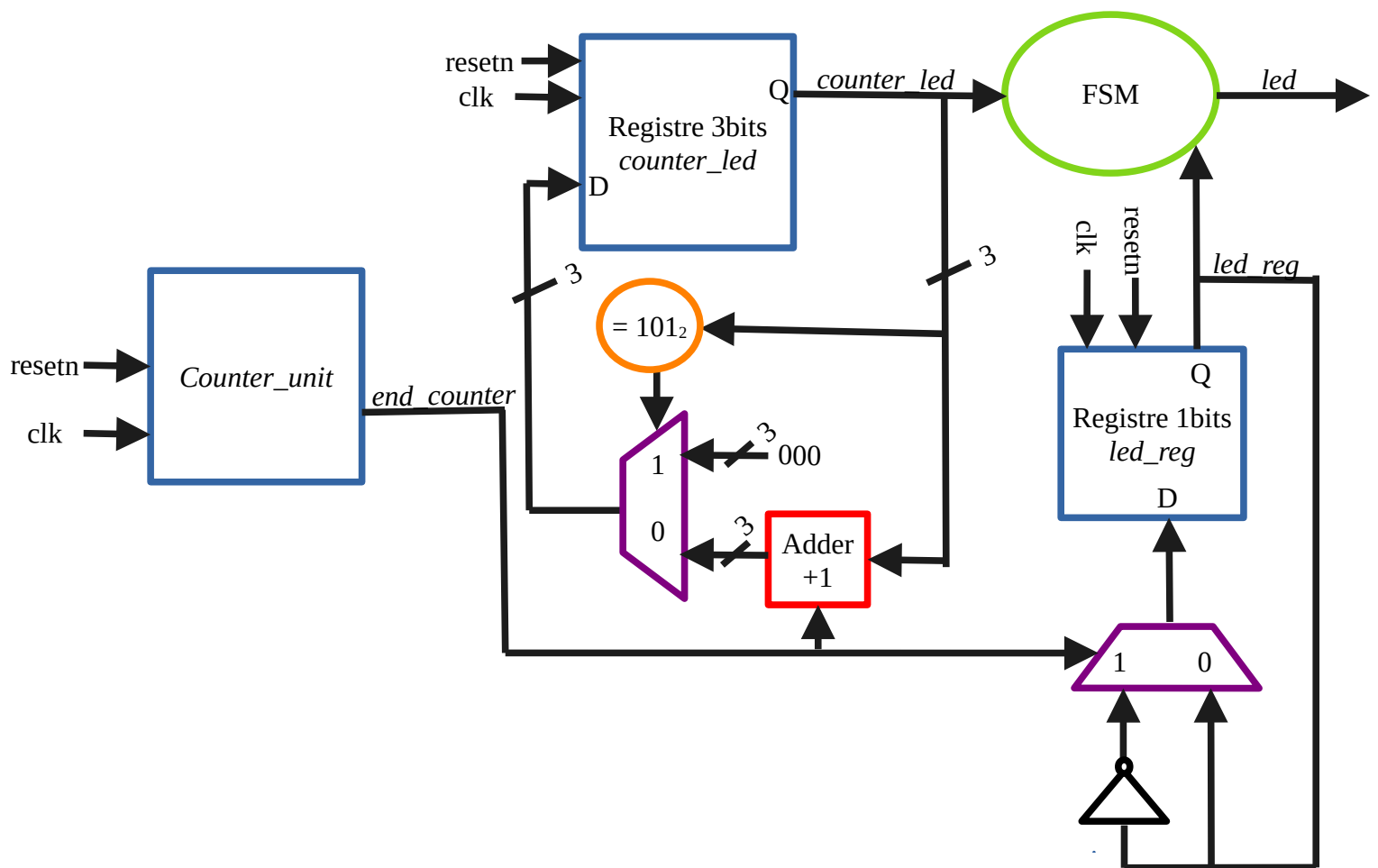
TP3

Machine à états (FSM)

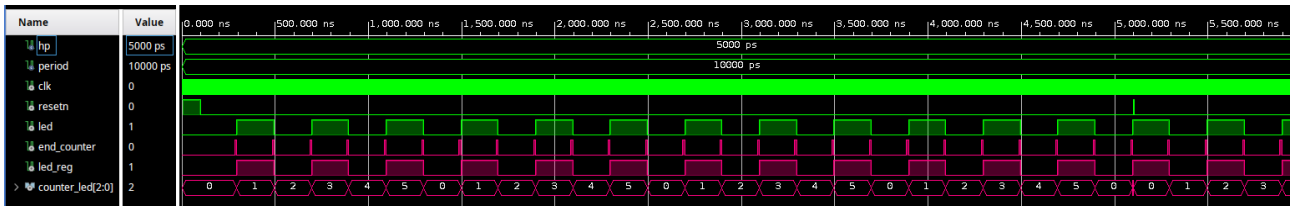
On cherche à construire ici une architecture permettant de faire clignoter des leds. Une machine d'état sera mise en œuvre pour commander les leds.

2)

Voici le schémas RTL de la structure permettant de faire clignoter une led.



4)



En vert les signaux tu testbench et en rose les signaux internes au *tp_fsm*.

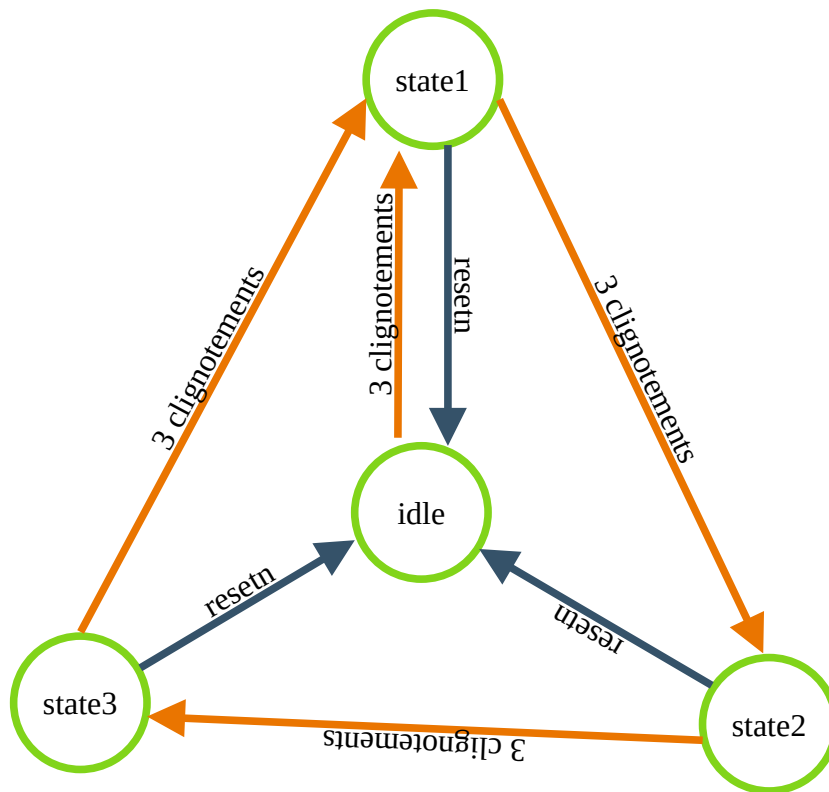
end_counter est à 1 pendant un cycle d'horloge tous les 20 cycles.

Lorsque celui-ci est à 1, *led_reg* et *led* changent d'état. Le compteur de led s'incrémente également.

counter_led compte le nombre de pulsation de *end_counter*. Lorsqu'il arrive à 5, soit 6 changement d'état de la led, il se remet à 0.

À 5100ns on envoi un reset pour tester la remise à 0 de l'architecture.

5)



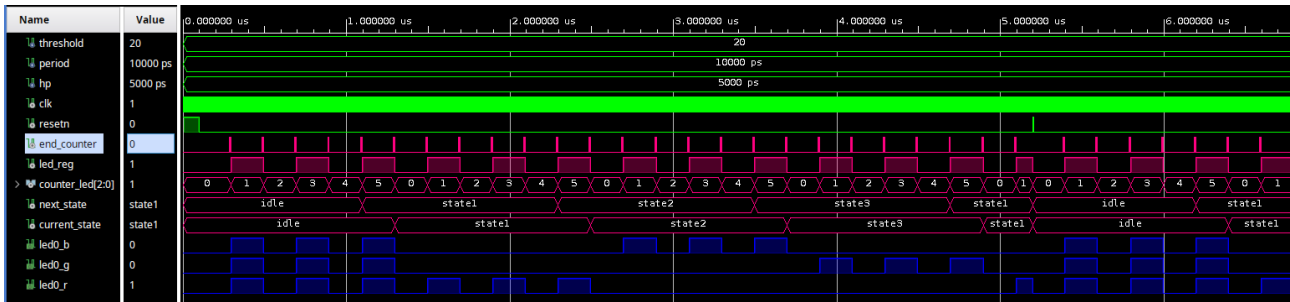
6)

Les signaux d'entrée sont : *clk* et *resetn*

Les signaux internes sont : *current_state*, *next_state*, *end_counter*, *counter_led* et *led_reg*

Les signaux de sortie sont : *led0_b*, *led0_g* et *led0_r*

8)



current_state et *next_state* commencent bien à idle après le reset.

Au 5^{ème} changement de la led, *next_state* passe à state1

Au 6^{ème} changement de la led, *current_state* passe à state1.

La led a donc bien clignoté 3 fois, soit 3 fois à 0 et 3 fois à 1.

On voit que la séquence d'état est correct : idle, stat1, state2, state3, state1, ...

Pour chaque état la led clignote 3 fois.

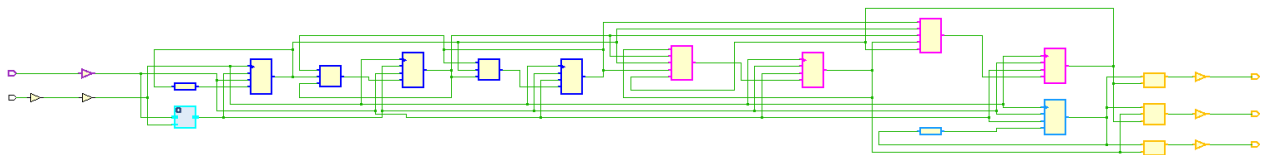
Le reset à 5200ns permet de vérifier la remise à 0.

current_state et *next_state* sont ramenés à idle.

En bleu les 3 signaux de led clignotent 3 fois par état. Ils suivent la séquence de couleur blanc, rouge, bleu, vert, rouge, ...

Le reset à 5200ns remets 3 signaux de led à la couleur blanche.

9)



Les entrées *clk* et *resetn* à gauche.

En cyan à gauche *counter_unit*.

En bleu foncé à gauche, Les registres et LUT du compteur de led

En rose les registres et LUT de la machine d'état. Ceux-ci stockent et manipulent *current_state*.

En bleu claire à droite, la LUT et le registre de *led_reg*.

En orange à droite, les 3 LUT des 3 signaux de led : *led0_b*, *led0_g* et *led0_r*.

7. Primitives

Ref Name	Used	Functional Category
FDCE	11	Flop & Latch
LUT5	5	LUT
OBUF	3	IO
LUT3	3	LUT
LUT2	3	LUT
LUT1	3	LUT
IBUF	2	IO
LUT4	1	LUT
BUFG	1	Clock

Pour l'exercice, j'ai réduit le *threshold* à 20. Le *counter_time* a besoin de 5 bits pour compter jusqu'à 20. 5 registres sont donc alloués à *counter_time*.

2 registres sont alloués à *current_state* pour lui permettre de stocker 4 états.

Il y a un registre pour *led_reg*.

Le compteur de led à besoin de 3 registres pour compter jusqu'à 5.

On retrouve bien les 11 registres affichés dans le rapport de synthèse.

Il y a 3 buffers de sortie pour *led0_b*, *led0_g*, *led0_r* et 2 buffers d'entrée pour *resetn* et *clk*.

L'horloge a aussi un buffer spécial.

Le rapport nous montre que 15 LUT sont utilisées.

Il y a une LUT par registre soit 11 LUT, une LUT pour chacune des 3 sortie et une LUT pour la sortie de *counter_time*.

Vivado adapte la taille des LUT en fonction du besoin, d'où les 5 tailles différentes.

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	8	0	17600	0.05
LUT as Logic	8	0	17600	0.05
LUT as Memory	0	0	6000	0.00
Slice Registers	11	0	35200	0.03
Register as Flip Flop	11	0	35200	0.03
Register as Latch	0	0	35200	0.00
F7 Muxes	0	0	8800	0.00
F8 Muxes	0	0	4400	0.00

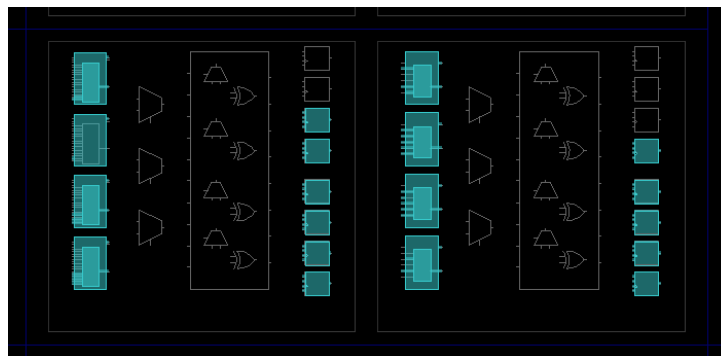
Pourtant seulement 8 « Slice LUTs » sont utilisées.

Après avoir regardé la documentation du Zynq-7000, je pense comprendre que chaque slice a 4 LUT. Chacune d'elles peut être utilisée comme une seule LUT6 ou comme 2 LUT5.

Lors de la synthèse, Vivado utilise donc 15 LUT5 réparties sur 2 slices.

Dans le rapport d'utilisation des slices, ne s'affichent que les LUT6 utilisées, donc seulement 8 LUT des 2 slices.

Les LUT sont en réalité utilisées comme des LUT5 et non des LUT6. On ne peut voir cela que dans les détails d'utilisation des LUT au point « 7.Primitive » du rapport.



Les LUT sont à gauche des slices et les registres à droite.

10)

Dans le fichier de contrainte, on configure l'horloge à 100MHz.

```
set_property -dict { PACKAGE_PIN H16 IOSTANDARD LVCMOS33 } [get_ports { clk }];  
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 4} [get_ports { clk }];
```

On connecte les 3 signaux de LED aux sorties de notre architecture.

```
set_property -dict { PACKAGE_PIN L15 IOSTANDARD LVCMOS33 } [get_ports { led0_b }];  
set_property -dict { PACKAGE_PIN G17 IOSTANDARD LVCMOS33 } [get_ports { led0_g }];
```

`set_property -dict { PACKAGE_PIN N15 IOSTANDARD LVCMOS33 } [get_ports { led0_r }];`

On connecte un bouton au signal d'entrée `resetrn`.

`set_property -dict { PACKAGE_PIN D20 IOSTANDARD LVCMOS33 } [get_ports { resetn }];`

11)

Design Timing Summary

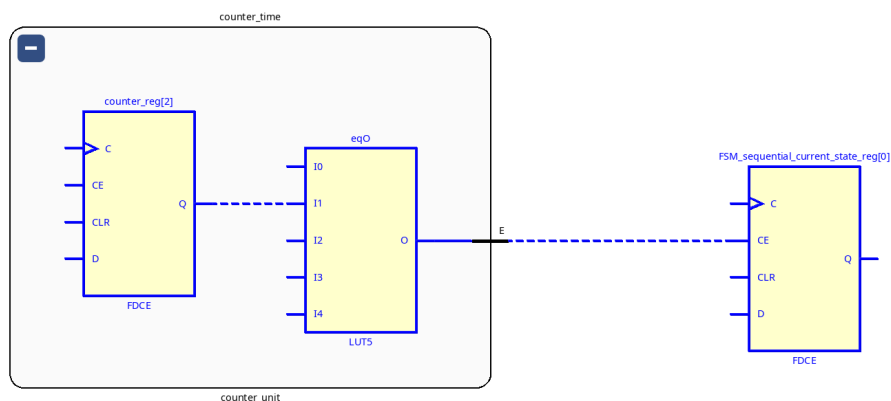
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 7,714 ns	Worst Hold Slack (WHS): 0,230 ns	Worst Pulse Width Slack (WPWS): 3,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 17	Total Number of Endpoints: 17	Total Number of Endpoints: 12

All user specified timing constraints are met.

Le TNS et le THS sont à 0ns.

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement
Path 1	7.714	1	6	counter_time/counter_reg[2]/C	FSM_sequential_...state_reg[0]/CE	2.060	0.580	1.480	10.0

Le chemin critique a un slack de 7, 714 ns et un délai 2,060ns. La période de l'horloge étant de 10ns.



Le chemin critique provient d'un registre de `counter_time` et arrive à un registre de la machine d'état.

Lorsqu'il sort de `counter_time`, le chemin critique correspond au signal `end_counter`.

12) Le bitstream est généré sans erreur mais je n'ai pas la carte pour tester l'architecture.